RescueNet

Guide complet du projet

Architecture microservices — Eureka, Gateway, OpenFeign, RabbitMQ, Keycloak, Docker

Dispatch intelligent des urgences médicales
Module « Applications Web Distribuées » — 4e année

Dépôt : github.com/ahmed4gaddes/-RescueNet-microservices

Sommaire

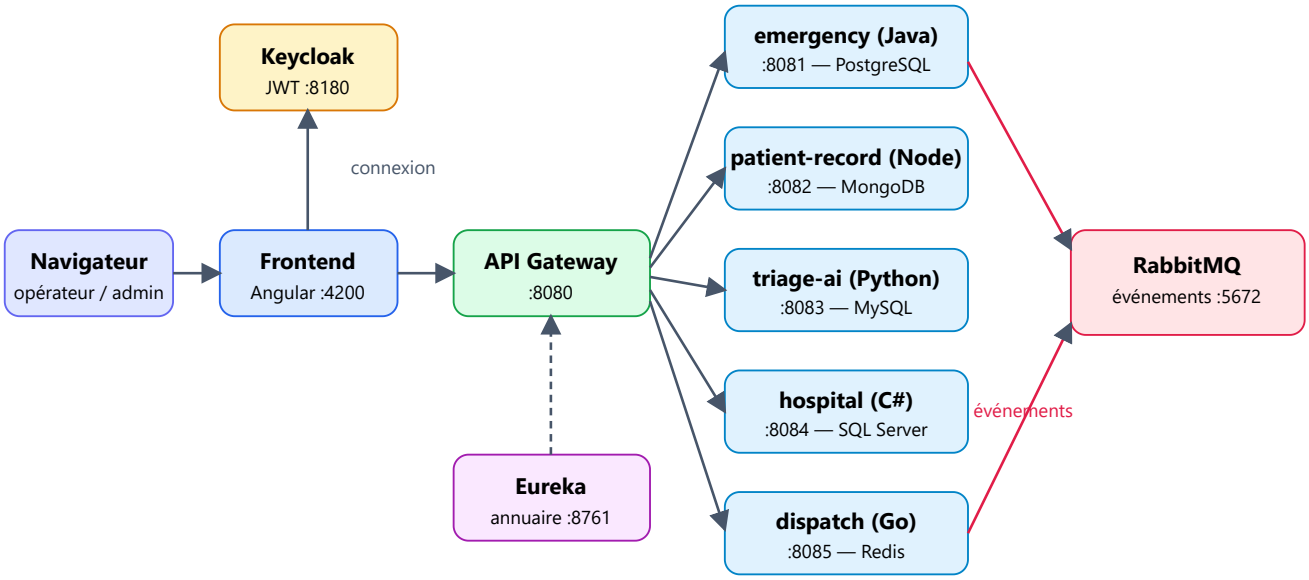
1. Vue d'ensemble du projet
2. Tous les ports disponibles
3. Les serveurs et leur rôle
4. Eureka — le Discovery Server
5. L'API Gateway
6. OpenFeign — appels entre microservices
7. Le point oublié : ce qui existe seulement en Java, sans port ni URL
8. Keycloak — authentification et JWT
9. Les rôles OPERATEUR et ADMIN
10. Les bases de données de chaque service
11. RabbitMQ — communication asynchrone
12. Swagger et OpenAPI
13. Docker : pourquoi une image par service
14. Docker Compose : pourquoi dans les microservices
15. Cloner et lancer le projet (pour vos amis)
16. Pousser le travail sur GitHub
17. Annexe : commandes utiles et dépannage

> Les chemins de fichiers sont donnés à partir de la racine du projet (`G:\project2`). Les numéros de ligne correspondent à la version du dépôt au moment de la rédaction : ils peuvent légèrement bouger si le code évolue.

1. Vue d'ensemble du projet

RescueNet coordonne une urgence médicale. Un opérateur du centre d'appels saisit l'appel ; la plateforme enchaîne automatiquement : **dossier du patient → triage par intelligence artificielle → réservation d'un lit dans l'hôpital le plus proche → envoi de l'ambulance la plus proche**, avec un suivi GPS simulé jusqu'à l'hôpital.

Le projet est une architecture **microservices polyglotte** : chaque service a son langage, sa base de données et son image Docker.



Flèches grises : appels HTTP. Flèche pointillée : enregistrement des services dans Eureka. Flèches rouges : messages RabbitMQ. `emergency-service` appelle aussi les 4 autres services avec OpenFeign (section 6).

Le déroulement d'une urgence

1. L'opérateur remplit le formulaire (écran « Nouvelle urgence » du frontend).
2. Le frontend envoie `POST /api/emergencies` avec un jeton JWT à l'API Gateway.
3. La Gateway vérifie le jeton et le rôle, trouve `EMERGENCY-SERVICE` dans Eureka et lui transmet la requête.
4. `emergency-service` orchestre (**saga**) : patient → triage → lit → ambulance, en appelant les autres services avec OpenFeign.
5. Si l'ambulance est indisponible : le lit déjà réservé est libéré (**compensation**) et la réponse est un 503 clair.
6. Le trajet de l'ambulance est simulé par `dispatch-service` qui publie des événements dans RabbitMQ ; à la fin, l'hôpital admet le patient, le dossier reçoit l'intervention et l'ambulance redevient libre.

2. Tous les ports disponibles

Applications

Port (hôte)	Composant	Usage
4200	Frontend Angular	Interface web — **http://localhost:4200**
8080	API Gateway	Point d'entrée de toutes les API (JWT obligatoire)
8081	emergency-service (Java)	Orchestration des urgences · Swagger `/swagger-ui.html`
8082	patient-record-service (Node.js)	Dossiers médicaux · Swagger `/swagger-ui`
8083	triage-ai-service (Python)	Triage par IA · Swagger `/docs`
8084	hospital-service (C#)	Hôpitaux et lits · Swagger `/swagger-ui`
8085	dispatch-service (Go)	Ambulances et GPS · Swagger `/swagger-ui`
8761	Eureka (discovery-server)	Annuaire — **http://localhost:8761**
8180	Keycloak	Authentification — **http://localhost:8180** (console `admin` / `admin123`)
9000	Keycloak (management)	Sonde de santé interne (`/health/ready`)

Infrastructure (bases de données et messagerie)

Port (hôte)	Composant	Utilisé par	Identifiants
5433 (5432 dans le conteneur)	PostgreSQL	emergency-service	`rescue` / `rescue123`, base `emergency_db`
27017	MongoDB	patient-record-service	sans mot de passe, base `patients`
3306	MySQL	triage-ai-service	`rescue` / `rescue123`, base `triage_db`
1433	SQL Server	hospital-service	`sa` / `Rescue_123!`, base `hospital_db`
6379	Redis	dispatch-service	sans mot de passe
5672	RabbitMQ (AMQP)	messages entre services	`rescue` / `rescue123`
15672	RabbitMQ (console web)	supervision	`rescue` / `rescue123`

> PostgreSQL est publié sur **5433** car un PostgreSQL installé sur la machine occupe souvent le port 5432. Dans le réseau Docker, les services se parlent par **nom** (ex. `postgres-emergency:5432`) : les ports de l'hôte ne servent qu'à y accéder depuis votre machine.

Où sont définis ces ports ? Applications : `docker-compose.yml` (sections `ports`) et `application.properties` ou le code de chaque service. Infrastructure : `docker-compose.infra.yml`.

3. Les serveurs et leur rôle

Serveur / service	Technologie	Rôle dans le projet	Dossier
discovery-server	Java, Spring Cloud Netflix Eureka	Annuaire : chaque service s'y enregistre ; les autres le trouvent par son nom.	`discovery-server/`
api-gateway	Java, Spring Cloud Gateway	Entrée unique, routage, vérification JWT et des rôles, CORS.	`api-gateway/`
emergency-service	Java 21, Spring Boot, OpenFeign	Chef d'orchestre : crée la mission et enchaîne patient, triage, lit, ambulance (saga avec compensation). Écoute les événements RabbitMQ.	`emergency-service/`
patient-record-service	Node.js, Express	Dossiers médicaux (allergies, maladies chroniques, historique des interventions).	`patient-record-service/`
triage-ai-service	Python, FastAPI, scikit-learn	Modèle d'IA (RandomForest) : gravité ROUGE / ORANGE / VERT et spécialité requise.	`triage-ai-service/`
hospital-service	C#, .NET 10	Hôpitaux, lits, réservations ; choisit l'hôpital le plus proche ayant la spécialité ; sortie automatique des patients après 3 minutes.	`hospital-service/`
dispatch-service	Go, Gin	Flotte d'ambulances : la plus proche (Redis GEOSEARCH) et simulation du trajet GPS.	`dispatch-service/`
frontend	Angular 22, Leaflet	Interface : tableau de bord, formulaire d'urgence, suivi sur carte, patients, hôpitaux, ambulances, triage.	`frontend/`
Keycloak	Docker (image officielle)	Serveur d'identité : connexion, comptes, rôles, émission des JWT.	`keycloak/`, `docker-compose.infra.yml`
RabbitMQ	Docker (image officielle)	Messagerie asynchrone entre services.	`docker-compose.infra.yml`
Bases de données	PostgreSQL, MongoDB, MySQL, SQL Server, Redis	Une base par service.	`docker-compose.infra.yml`

4. Eureka — le Discovery Server

Rôle

Eureka est l'**annuaire** des services. Au démarrage, chaque service dit à Eureka « je suis *TRIAGE-AI-SERVICE*, joignable à telle adresse », puis envoie un signal de vie (heartbeat) toutes les 10 secondes. Quand un service veut en appeler un autre, il demande à Eureka l'adresse à partir du **nom**. Résultat : **aucune URL ni port n'est écrit en dur** pour joindre un autre service, et un service arrêté disparaît de l'annuaire au bout de 30 secondes.

Emplacement dans le code

Élément	Fichier	Ce qu'il contient
Le serveur Eureka	`discovery-server/src/main/java/com/rescuenet/discoveryserver/DiscoveryServerApplication.java`	Ligne 7 : annotation `@EnableEurekaServer`

Configuration du serveur	`discovery-server/src/main/resources/application.properties`	`server.port=8761` ; il ne s'enregistre pas lui-même (lignes 6-7)
Dépendance	`discovery-server/pom.xml`	ligne 40 : `spring-cloud-starter-netflix-eureka-server`
Client Java (gateway, emergency)	`emergency-service/src/main/resources/application.properties`	ligne 22 : `eureka.client.service-url.defaultZone=http://localhost:8761/eureka/`
Client Node.js	`patient-record-service/src/server.js`	ligne 51 : bibliothèque `eureka-js-client`
Client Python	`triage-ai-service/app/main.py`	ligne 69 : `py-eureka-client` (`eureka_client.init_async`)
Client C#	`hospital-service/EurekaRegistration.cs`	client REST écrit à la main : register, heartbeat 10 s, deregister
Client Go	`dispatch-service/eureka.go`	client REST écrit à la main (même principe)

> En Docker, chaque service s'annonce à Eureka sous son **nom de conteneur** grâce à la variable `SERVICE_HOST` (ou `EUREKA_INSTANCE_HOSTNAME` pour Java), définie dans `docker-compose.yml`. En local, elle vaut `localhost` par défaut.

Comment le vérifier

Ouvrir **http://localhost:8761**. La section « Instances currently registered with Eureka » doit lister 6 applications en **UP** : API-GATEWAY, EMERGENCY-SERVICE, PATIENT-RECORD-SERVICE, TRIAGE-AI-SERVICE, HOSPITAL-SERVICE, DISPATCH-SERVICE.

5. L'API Gateway

Rôle

La Gateway est la **porte d'entrée unique** du système. Le frontend ne connaît qu'une adresse (`http://localhost:8080`) ; la Gateway :

- **route** chaque requête vers le bon service (`/api/emergencies/**` → emergency-service, etc.) en utilisant Eureka (`lb://NOM-DU-SERVICE`) ;
- **sécurise** : elle vérifie le jeton JWT et le rôle avant de transmettre (401 sans jeton, 403 si le rôle est insuffisant) ;
- gère le **CORS** (autorise le frontend sur le port 4200) ;
- **répartit la charge** si plusieurs instances d'un service existent.

Emplacement dans le code

Élément	Fichier	Détail
Application	`api-gateway/src/main/java/com/rescuenet/apigateway/ApiGatewayApplication.java`	classe de démarrage Spring Boot
Routes	`api-gateway/src/main/resources/application.properties`	lignes 15-35 : 5 routes `lb://EMERGENCY-SERVICE` , `PATIENT-RECORD-SERVICE` , `TRIAGE-AI-SERVICE` , `HOSPITAL-SERVICE` , `DISPATCH-SERVICE`
Sécurité (JWT, rôles, CORS)	`api-gateway/src/main/java/com/rescuenet/apigateway/SecurityConfig.java`	règles `pathMatchers` aux lignes 45-54 ; lecture des rôles Keycloak (`realm_access.roles`)
Adresse de Keycloak	`api-gateway/src/main/resources/application.properties`	lignes 38-39 : `issuer-uri` et `jwk-set-uri`
Dépendances	`api-gateway/pom.xml`	ligne 40 : `spring-cloud-starter-gateway-server-webflux` ; ligne 44 : `spring-boot-starter-oauth2-resource-server`

Les routes

Chemin appelé	Service ciblé
`/api/emergencies/**`	EMERGENCY-SERVICE
`/api/patients/**`	PATIENT-RECORD-SERVICE
`/api/triage/**`	TRIAGE-AI-SERVICE
`/api/hospitals/**`	HOSPITAL-SERVICE
`/api/dispatch/**`	DISPATCH-SERVICE

Vocabulaire : « API server » désigne ici l'**API Gateway**, le serveur frontal par lequel passent toutes les requêtes. Si vous pensiez à l'annuaire des services, c'est le **Discovery Server (Eureka)** décrit en section 4.

6. OpenFeign — appels entre microservices

Rôle

OpenFeign permet à `emergency-service` d'appeler les autres services en déclarant une simple **interface Java**. On n'écrit ni URL, ni port, ni code HTTP : on écrit le **nom Eureka** du service et la signature de la méthode. La réponse JSON est convertie automatiquement en objet Java (**DTO**).

```
@FeignClient(name = "TRIAGE-AI-SERVICE", contextId = "triageClient")   // nom Eureka : aucune URL
public interface TriageClient {
    @PostMapping("/api/triage")
    TriageResponse triage(@RequestBody TriageRequest body);          // JSON <-> objets Java
}
```

Emplacement dans le code (tout dans emergency-service)

Élément	Fichier
Activation	`emergency-service/src/main/java/com/rescuenet/emergency/EmergencyServiceApplication.java` — ligne 8 : `@EnableFeignClients`
Les 4 clients	`emergency-service/src/main/java/com/rescuenet/emergency/client/` : `PatientClient`, `TriageClient`, `HospitalClient`, `DispatchClient`
Les DTO (7 objets)	`emergency-service/src/main/java/com/rescuenet/emergency/client/dto/`
Façade avec résilience	`…/client/FeignRemoteServices.java` — ligne 28 (classe), ligne 82 (méthode `call` : une nouvelle tentative, puis erreur 503)
Utilisation (la saga)	`…/mission/MissionService.java` — lignes 68 (triage), 77 (lit), 89 (ambulance), 159 (patient)
Timeouts	`emergency-service/src/main/resources/application.properties` — lignes 32-34 (2 s de connexion, 6 s de lecture)
Dépendance	`emergency-service/pom.xml` — ligne 60 : `spring-cloud-starter-openfeign`

> **Ce que cela apporte :** couplage faible (aucune adresse en dur), code court et lisible, DTO typés, intégration à Eureka et à la répartition de charge.
> **Limite :** OpenFeign n'existe qu'en Java ; les autres langages (Node, Python, C#, Go) utilisent leur propre client HTTP.

7. Le point oublié : ce qui existe seulement en Java, sans port ni URL

> **À confirmer :** vous avez écrit que vous ne retrouvez plus le nom d'une chose « qui se fait seulement en Java et qui n'a pas de port ni d'URL ». Je n'ai pas pu deviner avec certitude ; voici les deux candidats les plus probables, tous deux présents dans le projet. Si c'est autre chose, dites-le-moi et j'ajoute la section.

Candidat 1 — le client OpenFeign (« client déclaratif »)

C'est exactement la phrase de votre atelier : *« Déclarer un client OpenFeign et appeler un microservice par son nom Eureka, sans URL ni port écrits en dur. »* Il n'existe qu'en Java (Spring Cloud) ; il n'a ni port ni URL, seulement un **nom** (`name = "TRIAGE-AI-SERVICE"`). Voir section 6.

Candidat 2 — Spring Cloud LoadBalancer (répartiteur de charge côté client)

C'est le composant qui transforme le nom `TRIAGE-AI-SERVICE` en vraie adresse. Il demande à Eureka la liste des instances et en choisit une (round-robin). Il travaille **à l'intérieur de l'application Java** : il n'a pas de port ni d'URL propres.

Élément	Emplacement
Dépendance	`emergency-service/pom.xml` — ligne 56 : `spring-cloud-starter-loadbalancer`
Route `lb://` de la Gateway	`api-gateway/src/main/resources/application.properties` — lignes 18-34 (le préfixe `lb://` = « passe par le LoadBalancer »)
Ancienne version (conservée)	`emergency-service/src/main/java/com/rescuenet/emergency/remote/HttpConfig.java` — ligne 28 : `@LoadBalanced`

Autres notions voisines, qui ne sont **pas** utilisées dans le projet : **Spring Cloud Config Server** (configuration centralisée) et **Resilience4j** (circuit breaker).

8. Keycloak — authentification et JWT

Rôle

Keycloak est le **serveur d'identité** : il connaît les utilisateurs, vérifie leurs mots de passe, leur attribue des **rôles** et émet des **jetons JWT** signés. Les services n'ont jamais accès aux mots de passe : la Gateway vérifie seulement la signature du jeton et lit les rôles qu'il contient.

Le circuit d'une connexion

1. Le navigateur ouvre le frontend ; `keycloak-js` redirige vers la page de connexion Keycloak (protocole OpenID Connect, **code + PKCE**).
2. L'utilisateur saisit son identifiant et son mot de passe **sur Keycloak**.
3. Keycloak renvoie un code ; le frontend l'échange contre un **JWT** (valable 15 minutes, renouvelé automatiquement).
4. Chaque appel API porte l'en-tête `Authorization: Bearer <jwt>`.
5. La Gateway vérifie la signature (clés publiques de Keycloak), lit `realm_access.roles` et applique les règles par rôle.

Emplacement dans le code

Élément	Fichier	Détail
Conteneur Keycloak	`docker-compose.infra.yml`	lignes 105-126 : image `keycloak:26.0`, port 8180, import du realm
Realm, rôles, comptes, clients	`keycloak/rescuenet-realm.json`	realm `rescuenet` ; rôles OPERATEUR et ADMIN (lignes 12-20) ; client `rescuenet-frontend` (ligne 25) ; comptes `operateur` et `admin`
Sécurité côté serveur	`api-gateway/…/SecurityConfig.java`	règles par rôle, lecture du JWT
Adresse de Keycloak (Gateway)	`api-gateway/src/main/resources/application.properties`	lignes 38-39
Connexion côté Angular	`frontend/src/app/core/auth.service.ts`	ligne 11 : `new Keycloak(...)` ; ligne 22 : `init({ onLoad: 'login-required', pkceMethod: 'S256' })`
Ajout du jeton aux appels	`frontend/src/app/core/auth.interceptor.ts`	ligne 12 : en-tête `Authorization: Bearer`
Protection des pages	`frontend/src/app/core/role.guard.ts, app.routes.ts`	route `/triage` réservée à ADMIN (ligne 13 de `app.routes.ts`)
Démarrage au lancement	`frontend/src/app/app.config.ts`	`provideAppInitializer` déclenche la connexion

Comment le mettre en place soi-même (de zéro)

Méthode manuelle (pour comprendre) — console Keycloak, **http://localhost:8180**, compte `admin` / `admin123` :

1. Lancer Keycloak (`docker compose -f docker-compose.infra.yml up -d keycloak`).
2. Menu en haut à gauche : **Create realm** → nom `rescuenet` → Create.
3. **Realm roles** → Create role : `OPERATEUR`, puis `ADMIN`.
4. **Clients** → Create client : Client ID `rescuenet-frontend`, type OpenID Connect ; « Client authentication » **désactivé** (client public) ; « Standard flow » coché.
5. Dans ce client : **Valid redirect URIs** = `http://localhost:4200/*` ; **Web origins** = `http://localhost:4200` ; onglet Advanced → **PKCE Code Challenge Method** = `S256`.
6. **Users** → Add user (`operateur`, `admin`), onglet **Credentials** → Set password (« Temporary » désactivé), onglet **Role mapping** → Assign role.
7. Côté Gateway : renseigner `issuer-uri` et `jwk-set-uri` (voir tableau) et écrire les règles dans `SecurityConfig.java`.
8. Côté Angular : installer `keycloak-js` (`npm i keycloak-js`), écrire `auth.service.ts`, l'intercepteur et le guard.

Méthode automatique utilisée dans ce projet : tout le realm (rôles, comptes, client) est décrit dans `keycloak/rescuenet-realm.json` et importé au démarrage grâce à la commande `start-dev --import-realm` du conteneur. Vos amis n'ont donc **rien à configurer** : les deux comptes existent dès le premier lancement.

Compte	Mot de passe	Rôles
`operateur`	`operateur123`	OPERATEUR
`admin`	`admin123`	ADMIN (+ OPERATEUR)

> Ce sont des comptes de **démonstration**. Keycloak n'importe le realm que si sa base est vide : si vous modifiez le fichier JSON, supprimez le volume (`docker compose down -v`) pour le réimporter. Le client `rescuenet-cli` (connexion directe par mot de passe) sert uniquement aux tests Postman : il est à retirer en production.

9. Les rôles OPERATEUR et ADMIN

Qui est qui

- **OPERATEUR** : employé du centre d'appels d'urgence. Il décroche, saisit l'urgence dans l'application et suit la mission. Ce n'est ni le citoyen (qui appelle par téléphone), ni une application propre à un hôpital.
- **ADMIN** : responsable de la plateforme. Il a tous les droits de l'opérateur, plus l'accès au triage IA et la modification des dossiers patients.

Droits de chaque rôle (appliqués par la Gateway)

Action	OPERATEUR	ADMIN
Créer une urgence (`POST /api/emergencies`)	✓	✓
Suivre les missions (`GET /api/emergencies/**`)	✓	✓
Consulter patients, hôpitaux, ambulances (`GET`)	✓	✓
Créer / modifier un dossier patient (`POST` , `PUT /api/patients/**`)	✗ (403)	✓
Triage IA (`/api/triage/**`)	✗ (403)	✓
Actuator, lien Eureka dans le menu	✗	✓
Sans jeton	401 (non authentifié)	

Dans l'interface : l'opérateur ne voit pas le menu « Triage IA » ni le bouton « Modifier le dossier » ; s'il tape l'adresse `/triage` , il obtient la page « Accès refusé ». Les règles sont écrites à `api-gateway/…/SecurityConfig.java` (lignes 45-54) et dans `frontend/src/app/app.routes.ts`.

10. Les bases de données de chaque service

Règle d'or des microservices : **une base par service**, jamais d'accès croisé. Chaque base est choisie pour son usage.

Service	Base	Pourquoi ce choix	Où la connexion est configurée	Où sont les modèles de données
emergency-service	**PostgreSQL** (5433)	missions et statuts : données relationnelles, transactions	`emergency-service/src/main/resources/application.properties` lignes 5-7	`…/mission/Mission.java (ligne 14),` `MissionRepository`
patient-record-service	**MongoDB** (27017)	dossier médical : document flexible avec listes et historique imbriqué	`patient-record-service/src/server.js ligne 12`	`patient-record-service/src/models.js (lignes 3 et 14)`
triage-ai-service	**MySQL** (3306)	historique des triages (écosystème Python)	`triage-ai-service/app/main.py ligne 23`	`triage-ai-service/app/main.py (classe` `TriageRecord` `, ligne 35)`
hospital-service	**SQL Server** (1433)	lits : décrément atomique, forte cohérence	`hospital-service/Program.cs ligne 10`	`hospital-service/Data.cs (` `Hospital` `ligne 5,` `BedReservation` `ligne 21)`
dispatch-service	**Redis** (6379)	flotte et positions GPS : `GEOSEARCH` très rapide	`dispatch-service/main.go ligne 42`	`dispatch-service/store.go (ligne 115 : recherche géographique)`

Où les bases sont lancées : `docker-compose.infra.yml` (un conteneur par base, avec un volume pour conserver les données). En Docker, les adresses sont remplacées par des variables d'environnement dans `docker-compose.yml` (par exemple `SPRING_DATASOURCE_URL` , `MONGO_URL` , `MYSQL_URL` , `ConnectionStrings__Hospital` , `REDIS_ADDR`) afin de pointer vers le **nom du conteneur** au lieu de `localhost` .

11. RabbitMQ — communication asynchrone

Rôle

Les appels OpenFeign sont **synchrones** : on attend la réponse. RabbitMQ sert aux situations où l'on n'attend rien : un service **publie un événement**, d'autres **le consomment** quand ils le peuvent. Le producteur ne connaît pas les consommateurs : le couplage est très faible. Si un consommateur est arrêté, le message attend dans sa file.

Vocabulaire

Terme	Dans le projet
Producer	celui qui publie (dispatch, emergency)

Exchange (de type *topic*)	`rescuenet.events` : reçoit les messages et les aiguille
Routing key	l'étiquette du message : `mission.status.changed` , `mission.completed` , `mission.dispatched`
Queue	file d'attente propre à chaque consommateur
Consumer	celui qui lit sa file (emergency, hospital, patient-record, dispatch)

Les événements

Événement	Producteur	Consommateurs	Effet
`mission.status.changed` (ON_SCENE, TRANSPORTING, ARRIVED)	dispatch-service	emergency-service (file `emergency.status-changed`)	met à jour le statut de la mission
`mission.completed`	emergency-service	hospital (`hospital.mission-completed`), patient-record (`patient-record.mission-completed`), dispatch	admission du patient, intervention ajoutée au dossier, ambulance libérée
`mission.dispatched`	emergency-service	informatif	—

Emplacement dans le code

Élément	Fichier
Le serveur RabbitMQ	`docker-compose.infra.yml — ligne 86 (service rabbitmq , ports 5672 et 15672)`
Exchange, file, liaison (Java)	`emergency-service/src/main/java/com/rescuenet/emergency/events/RabbitConfig.java — lignes 13-28`
Publication (Java)	`…/events/MissionPublisher.java ( RabbitTemplate )`
Réception (Java)	`…/events/DispatchStatusListener.java ( @RabbitListener )`
Configuration de connexion	`emergency-service/src/main/resources/application.properties — lignes 12-19`
Go	`dispatch-service/events.go — Publish (ligne 50), consume (ligne 82)`
C#	`hospital-service/EventConsumer.cs (file hospital.mission-completed )`
Node.js	`patient-record-service/src/events.js (file patient-record.mission-completed )`

Voir les messages : console RabbitMQ **http://localhost:15672** (`rescue` / `rescue123`) → onglets *Exchanges* et *Queues*.

12. Swagger et OpenAPI

Rôle

OpenAPI est un standard qui décrit une API REST dans un fichier (JSON) : routes, paramètres, formats, codes de réponse. **Swagger UI** est la page web qui lit ce fichier et permet de **lire la documentation et tester les routes dans le navigateur**, sans Postman. (Vous avez écrit « open IA » : il s'agit bien d'**OpenAPI**, sans lien avec l'intelligence artificielle.)

Comment se connecter à Swagger

Il n'y a **pas de compte à saisir** : on ouvre simplement l'adresse du service dans le navigateur.

Service	Adresse Swagger UI	Fichier OpenAPI (JSON)
emergency-service	**http://localhost:8081/swagger-ui.html**	`/v3/api-docs`
patient-record-service	**http://localhost:8082/swagger-ui**	`/v3/api-docs`
triage-ai-service	**http://localhost:8083/docs**	`/openapi.json`
hospital-service	**http://localhost:8084/swagger-ui**	`/v3/api-docs`
dispatch-service	**http://localhost:8085/swagger-ui**	`/v3/api-docs`

Tester une route

1. Ouvrir l'adresse (par exemple celle d'emergency-service).
2. Cliquer sur une route (ex. `POST /api/emergencies`) pour la déplier.
3. Cliquer **Try it out**, modifier le JSON d'exemple (copier `scripts/demo-emergency.json`).
4. Cliquer **Execute** : la réponse (code, corps JSON) s'affiche en dessous.

> Swagger appelle le service **directement** (ports 8081 à 8085), sans passer par la Gateway : il n'y a donc pas de contrôle de jeton ni de rôle. C'est pratique pour la démonstration. Pour tester les **rôles** (401, 403) via la Gateway, utilisez la collection Postman `docs/postman/`. Les interfaces des services Node, C# et Go chargent leurs fichiers depuis `unpkg.com` : une connexion internet est nécessaire.

Emplacement dans le code

Service	Fichiers
emergency (Java)	`emergency-service/pom.xml` ligne 63 (`springdoc-openapi-starter-webmvc-ui`) ; `application.properties` lignes 44-45 ; annotations `@Tag` / `@Operation` dans `…/web/EmergencyController.java`
patient-record (Node)	`patient-record-service/src/openapi.js` (la spécification) ; `src/server.js` ligne 44 (`swagger-ui-express`)
triage-ai (Python)	natif dans FastAPI : `triage-ai-service/app/main.py` ligne 84 (`FastAPI(...)`) — aucune configuration supplémentaire
hospital (C#)	`hospital-service/Program.cs` lignes 11 (`AddOpenApi`), 29 (`MapOpenApi`), 30 (page `/swagger-ui`)
dispatch (Go)	`dispatch-service/openapi.go` (fonction `registerDocs` , ligne 10)

13. Docker : pourquoi une image par service

Le problème que Docker résout

Ce projet utilise **cinq langages** (Java, Node.js, Python, C#, Go) et six technologies de données. Sans Docker, chaque ami devrait installer JDK 21, Node 22, Python 3.12, .NET 10, Go, Maven, puis configurer chaque base, et le projet « marcherait sur ma machine » seulement. **Une image Docker contient le programme *et* tout ce dont il a besoin** : le résultat est le même sur toutes les machines.

Pourquoi une image *par service*

- **Indépendance** : on reconstruit, redémarre ou met à l'échelle un service sans toucher aux autres (principe du déploiement indépendant des microservices).
- **Isolation** : chaque service a ses propres dépendances et versions, sans conflit.
- **Isolation des pannes** : si un conteneur plante ou est arrêté, les autres continuent (démo : `docker stop rescuenet-dispatch`).
- **Images légères** : Dockerfile en **deux étapes** (construction puis exécution) : l'image finale ne contient pas les outils de compilation (ex. dispatch : 55 Mo, frontend : 75 Mo).
- **Reproductibilité** : même version partout, ce qui facilite les tests et la soutenance.

Les 8 Dockerfile

Dossier	Base de construction → base d'exécution	Particularité
`discovery-server/Dockerfile`	temurin-21-jdk → temurin-21-jre	sonde de santé (`HEALTHCHECK`) utilisée par Compose
`api-gateway/Dockerfile`	temurin-21-jdk → temurin-21-jre	—
`emergency-service/Dockerfile`	temurin-21-jdk → temurin-21-jre	compile avec `mvnw` (Maven inclus au projet)
`patient-record-service/Dockerfile`	node:22-alpine	`npm ci --omit=dev` , utilisateur non root
`triage-ai-service/Dockerfile`	python:3.12-slim	entraîne le modèle d'IA **pendant la construction** (`train_model.py`)
`hospital-service/Dockerfile`	dotnet/sdk:10.0 → dotnet/aspnet:10.0	—
`dispatch-service/Dockerfile`	golang:1.27-alpine → alpine:3.20	binaire statique, image minuscule
`frontend/Dockerfile (+ nginx.conf)`	node:22-alpine → nginx:alpine	nginx sert l'application Angular compilée

Les fichiers `.dockerignore` de chaque dossier empêchent de copier dans l'image les dossiers inutiles (`node_modules` , `target` , `.venv` , `bin` , `obj` …).

14. Docker Compose : pourquoi dans les microservices

Rôle

Avec 8 services applicatifs et 7 éléments d'infrastructure, soit **15 conteneurs**, les lancer un par un avec `docker run` serait long et source d'erreurs. **Docker Compose** décrit l'ensemble dans un fichier et le démarre en **une seule commande** : `docker compose up -d --build` .

Ce que Compose apporte

- **Un réseau commun** : les conteneurs se joignent par leur **nom** (`discovery-server` , `rabbitmq` , `postgres-emergency` …), sans adresse IP.
- **L'ordre de démarrage** (`depends_on` + `healthcheck`) : bases et RabbitMQ d'abord, puis Eureka, puis les services, puis le frontend.
- **La configuration** (variables d'environnement) : où se trouve chaque base, quel Eureka utiliser.
- **Les volumes** : les données des bases survivent à un redémarrage.
- **Les ports publiés** : identiques pour tout le monde (4200, 8080, 8761…).
- **Des commandes simples** pour tout arrêter, voir l'état, lire les logs.

Les deux fichiers

Fichier	Contenu
`docker-compose.infra.yml`	7 conteneurs d'infrastructure : PostgreSQL, MongoDB, MySQL, SQL Server, Redis, RabbitMQ, Keycloak
`docker-compose.yml`	les 8 applications ; il **inclut** le fichier d'infrastructure (ligne 12 : `include:`)

> Deux modes : **Docker** (tout en conteneurs, `docker compose up -d --build`) ou **développement** (infrastructure en Docker + services en processus locaux avec `scripts\start-all.ps1`). Ne pas utiliser les deux en même temps : les ports se chevauchent.

15. Cloner et lancer le projet (pour vos amis)

Avec cette méthode, vos amis obtiennent **exactement les mêmes ports que vous** : ils sont fixés dans `docker-compose.yml`. Rien à installer à part Git et Docker Desktop (pas besoin de Java, Node, Python, .NET ni Go).

Étape 0 — Prérequis

Outil	Détail
Git	https://git-scm.com/downloads
Docker Desktop	https://www.docker.com/products/docker-desktop — sous Windows, activer **WSL 2** (proposé à l'installation)
Mémoire	au moins **6 Go de RAM libres** ; donner **5 Go ou plus** à Docker (Docker Desktop → Settings → Resources → Memory)
Espace disque	environ 10 Go (images et conteneurs)
Internet	indispensable au premier lancement (téléchargement des images) et pour les cartes
Ports libres	4200, 8080-8085, 8761, 8180, 9000, 5433, 27017, 3306, 1433, 6379, 5672, 15672 (fermer tout autre programme qui les utilise)

> **Mac avec puce Apple (M1/M2/M3)** : l'image SQL Server n'existe qu'en architecture Intel ; il faut activer « Use Rosetta for x86_64/amd64 emulation » dans Docker Desktop. Je n'ai pas testé le projet sur Mac : prévoyez un peu de dépannage.

Étape 1 — Cloner le dépôt

Ouvrir un terminal (PowerShell, Terminal macOS ou Git Bash) dans le dossier de votre choix :

```
git clone -b feature/docker-keycloak-angular https://github.com/ahmed4gaddes/-RescueNet-microservices.git RescueNet
cd RescueNet
```

Le travail se trouve pour l'instant sur la branche `feature/docker-keycloak-angular`. Une fois cette branche fusionnée dans `main`, la commande simplifiée suffira : `git clone https://github.com/ahmed4gaddes/-RescueNet-microservices.git RescueNet`. Le dossier final `RescueNet` est choisi exprès : le nom du dépôt commence par un tiret, ce qui gêne la commande `cd`.

Si le dépôt est privé, GitHub demandera de vous connecter (le propriétaire doit ajouter vos comptes comme collaborateurs : Settings → Collaborators).

Étape 2 — Démarrer Docker Desktop

1. Lancer **Docker Desktop** (menu Démarrer / Applications).
2. Attendre que l'état en bas à gauche indique **« Engine running »** (icône verte).
3. Vérifier dans le terminal : `docker info` ne doit afficher aucune erreur.

Étape 3 — Construire les images et lancer les serveurs

```
docker compose up -d --build
```

- **Première fois** : Docker télécharge les images de base et construit les 8 images de l'application. Comptez **de 10 à 40 minutes** selon la connexion.
- **Les fois suivantes** : 1 à 2 minutes (tout est en cache).
- Docker démarre les 15 conteneurs dans le bon ordre (bases, RabbitMQ, Keycloak, puis Eureka, puis les services, puis le frontend).

Suivre l'avancement :

```
docker compose ps              # état des 15 conteneurs (Up / healthy)
docker compose logs -f emergency-service    # logs d'un service (Ctrl+C pour quitter)
```

Étape 4 — Vérifier les serveurs dans Eureka

Attendre environ une minute après le démarrage, puis ouvrir **http://localhost:8761**. On doit voir **6 applications en UP** :

API-GATEWAY · EMERGENCY-SERVICE · PATIENT-RECORD-SERVICE · TRIAGE-AI-SERVICE · HOSPITAL-SERVICE · DISPATCH-SERVICE

Étape 5 — Ouvrir le frontend et se connecter

Ouvrir **http://localhost:4200** : la page de connexion Keycloak s'affiche. Comptes :

Compte	Mot de passe	Rôle
`operateur`	`operateur123`	Opérateur
`admin`	`admin123`	Administrateur

Étape 6 — Scénario rapide à jouer

1. Se connecter en `operateur` → bouton **✚ Signaler une urgence** → **Remplir avec la démo** → 🚑 **Lancer le dispatch**.
2. Sur « Suivi de mission » : triage ROUGE, hôpital, ambulance qui bouge sur la carte, frise jusqu'à **Admis** (environ 15 secondes).
3. Menus **Missions**, **Patients**, **Hôpitaux**, **Ambulances** ; essayer `/triage` : « Accès refusé ».
4. Bouton **Se déconnecter**, puis se connecter en `admin` : menu **Triage IA**, bouton **Modifier le dossier** d'un patient.
5. Panne : `docker stop rescuenet-dispatch`, relancer une urgence → message « Service indispensable indisponible : DISPATCH-SERVICE » ; puis `docker start rescuenet-dispatch`.

Le scénario détaillé est dans `docs/SCENARIO-DEMO.md`.

Arrêter et relancer

```
docker compose down           # arrête et supprime les conteneurs (les données sont conservées)
docker compose up -d          # relance (sans reconstruire)
docker compose down -v        # arrête ET efface toutes les données (repartir de zéro)
```

Problèmes fréquents

Symptôme	Cause probable	Solution
`failed to connect to the docker API`	Docker Desktop n'est pas démarré	le lancer et attendre « Engine running »
`port is already allocated`	un autre programme utilise le port	fermer ce programme (ex. un PostgreSQL local sur 5432 n'est pas gênant : le projet utilise 5433)
Conteneurs qui s'arrêtent, machine lente	mémoire insuffisante	donner plus de RAM à Docker, fermer les autres applications
Page de connexion en erreur	Keycloak pas encore prêt	attendre 1 à 2 minutes, actualiser
Un service manque dans Eureka	démarrage en cours	patienter, puis `docker compose logs <service>`
Réponse 409 « Aucun lit disponible »	beaucoup d'urgences en peu de temps	attendre la sortie automatique (3 minutes) ou `docker compose down -v`
Cartes grises	pas d'internet	vérifier la connexion

16. Pousser le travail sur GitHub

Mon travail (déjà fait)

Le travail est poussé sur la branche `feature/docker-keycloak-angular` du dépôt **https://github.com/ahmed4gaddes/-RescueNet-microservices**. Pour qu'il soit la version par défaut, fusionnez la branche dans `main` : soit via le bouton **Compare & pull request** sur GitHub puis **Merge**, soit en ligne de commande :

```
git checkout main
git pull
git merge feature/docker-keycloak-angular
git push origin main
```

Pour un ami qui modifie le projet

```
git pull                                # récupérer les dernières modifications
git checkout -b feature/ma-fonctionnalite  # travailler sur sa propre branche
# ... modifier les fichiers ...
git status                              # voir ce qui a changé
git add .
git commit -m "Description claire de la modification"
git push -u origin feature/ma-fonctionnalite
# puis ouvrir une « Pull Request » sur GitHub pour la relire avant de fusionner
```

> Ne jamais ajouter au dépôt : `node_modules`, `target`, `logs/`, fichiers `.exe`, mots de passe réels. Le fichier `.gitignore` en exclut déjà la plupart.

17. Annexe : commandes utiles

Besoin	Commande

Tout démarrer (Docker)	`docker compose up -d --build`
État des conteneurs	`docker compose ps`
Logs d'un service	`docker compose logs -f <service>` (ex. `emergency-service`)
Redémarrer un service	`docker compose restart hospital-service`
Reconstruire un seul service	`docker compose up -d --build hospital-service`
Arrêter un conteneur (panne simulée)	`docker stop rescuenet-dispatch` puis `docker start rescuenet-dispatch`
Tout arrêter	`docker compose down`
Tout arrêter et effacer les données	`docker compose down -v`
Mode développement (sans conteneurs pour les services)	`docker compose -f docker-compose.infra.yml up -d` puis `powershell -ExecutionPolicy Bypass -File scripts\start-all.ps1` ; arrêt : `scripts\stop-all.ps1 -WithInfra`
Tests Postman en ligne de commande	`npx newman run docs\postman\RescueNet.postman_collection.json -e docs\postman\RescueNet.postman_environment.json`

Documents du dépôt

Fichier	Contenu
`README.md`	vue d'ensemble, installation, lancement
`docs/ARCHITECTURE.md`	diagrammes d'architecture, de séquence (saga), de déploiement et de sécurité
`docs/API.md`	routes, rôles, Swagger, événements RabbitMQ
`docs/JUSTIFICATION-ARCHITECTURE.md`	monolithe, SOA ou microservices ; limites ; choix d'outils
`docs/VEILLE-OBSERVABILITE.md`	équivalents d'Actuator pour Node.js et Python
`docs/SCENARIO-DEMO.md`	démarrage pas à pas et scénario par rôle
`docs/postman/`	collection et environnement Postman